

CS & IT ENGINEERING



Operating System

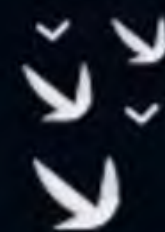
Deadlock

Lecture -1

By- Vishvadeep Gothi sir



Recap of Previous Lecture



Doubt ?

Topic

Reader-Writer Problem

Topic

Dining Philosopher Problem



Topics to be Covered



Topic

Deadlock

Topic

Deadlock Prevention

Topic

Deadlock Avoidance

Reader:-

wrt = 10

RC = 0
~~1~~
~~2~~
~~3~~
~~4~~
~~5~~
0

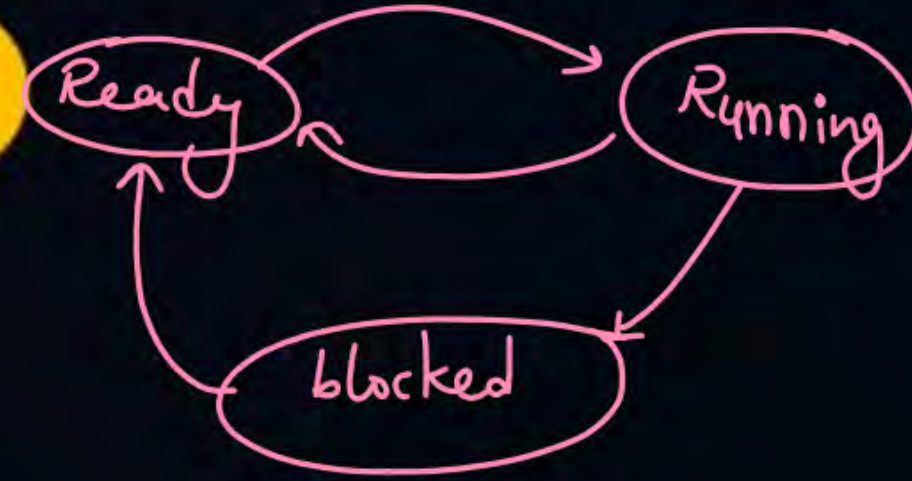
```
wait(mutex)
readcount++;
if(readcount == 1)
{ wait(wrt); }
signal(mutex)
// Reading
wait(mutex)
readcount--;
if(readcount == 0)
signal(wrt);
signal(mutex)
```

Writer:-

```
wait(wrt)
// writing
signal(wrt)
```



Topic : Operations on Resources



3 Operations on resources:

1. Request
2. Use (if OS assigned the resource to requesting process)
3. Release

→ if resource is not available $\Rightarrow$ process waits for it.



Topic : Deadlock



If two or more processes are waiting for such an event which is never going to occur

Process	Holds	waits
P1	Printer	keyboard Hard disk
P2	keyboard	Camera
P3	Hard disk	



Topic : Deadlock





Topic : Necessary Conditions for Deadlock



Deadlock can occur only when all following conditions are satisfied:

1. Mutual Exclusion
2. Hold & Wait
3. No-preemption $\Rightarrow$ Processes are not allowed to preempt resources from other processes.
4. Circular Wait



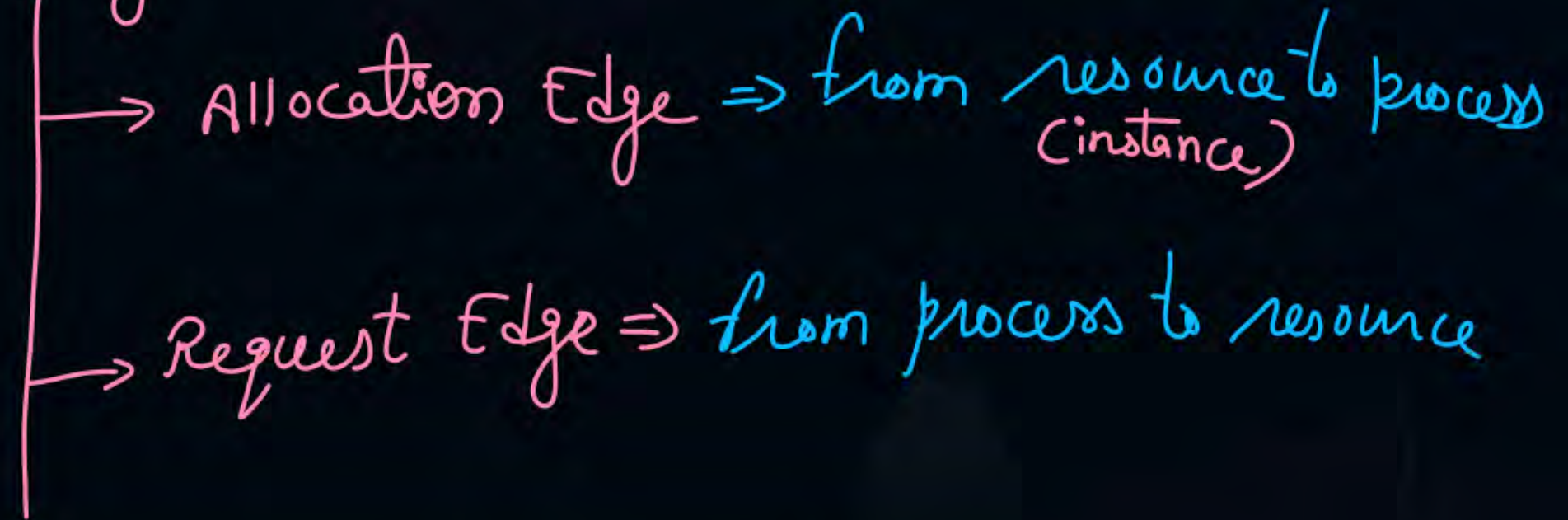
Topic : Resource Allocation Graph

(directed graph)

Vertices (nodes)

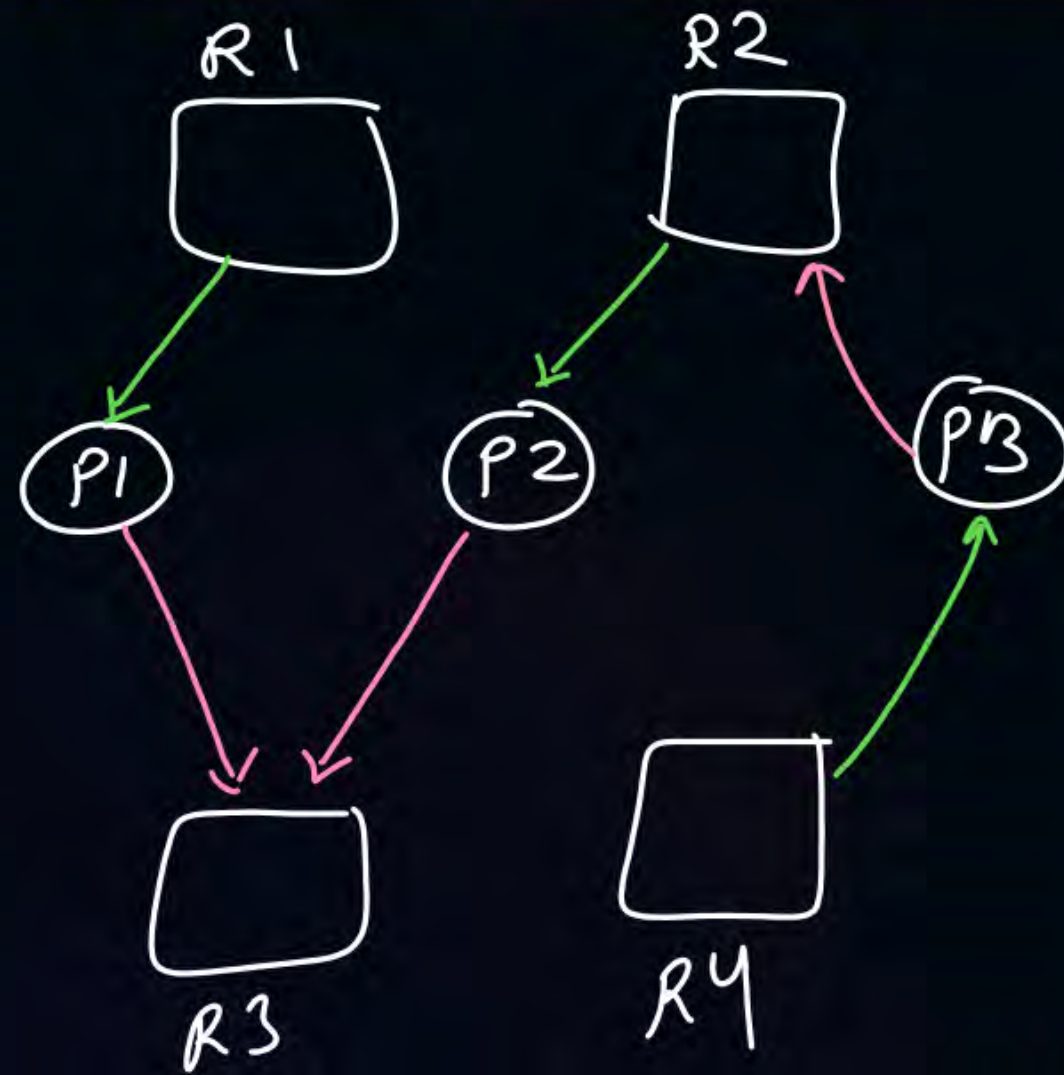


Edges



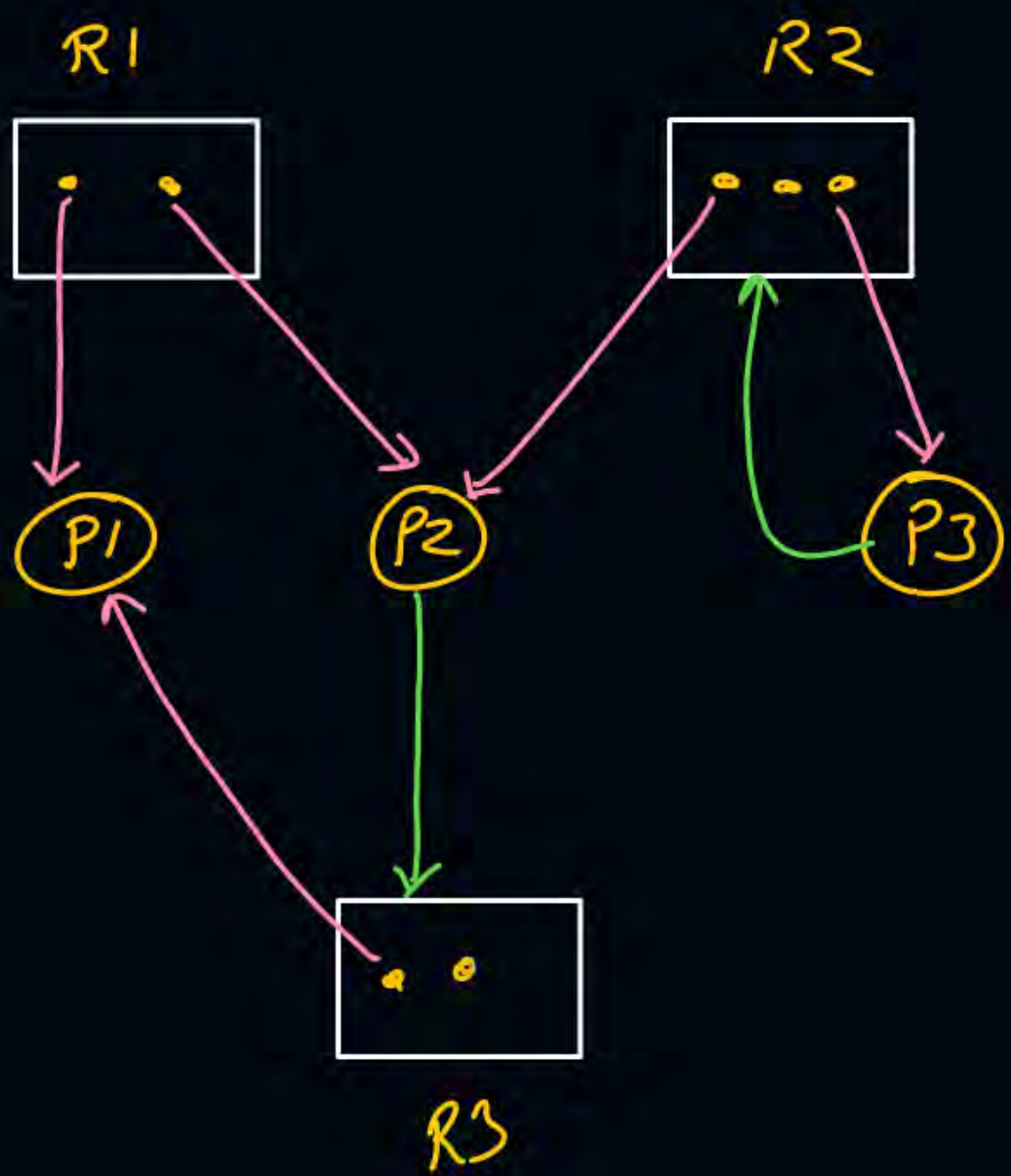


Topic : Resource Allocation Graph



if resources have multiple instances :-







Topic : Recovery From Deadlock

1. Make Sure that deadlock never occur
 - Prevent the system from deadlock or avoid deadlock

Deadlock prevention
Deadlock avoidance
2. Allow deadlock, detect and recover
3. Pretend that there is no any deadlock

all latest OS



Topic : Deadlock Prevention

try to prevent one of 4 necessary conditions to satisfy.

1. Preventing mutual Exclusion:-

→ make so many resources available in system such that there is no any waiting for resource needed. $\Rightarrow$ not practical

→ Have all processes as independent process in system. $\Rightarrow$ not practical

2. Preventing Hold and waits -

OS allows processes to either hold all their required resources or wait for all required resources.

→ Processes may suffer from starvation if all required resources are not available together.

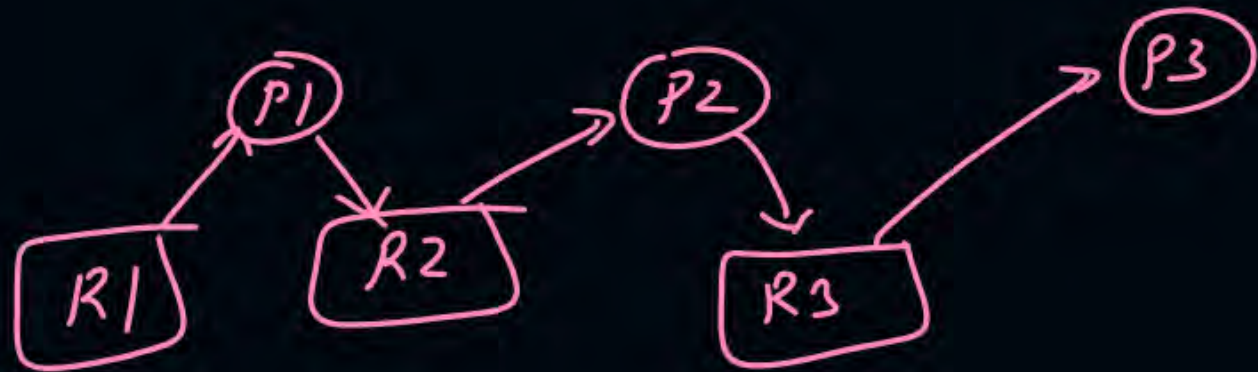
3. Preventing no-preemption:-

OS allows preemption resources.

- Preemption of which resource from which process (difficult decision)
- After preemption processes may go into unstable state.
- starvation

4. Preventing circular wait:-

Processes allowed to acquire resources only in increasing number order of resource number.



if P3 wants R1 also then P3 must release R3 and it will try to acquire R1 first then R3.



Topic : Deadlock Avoidance



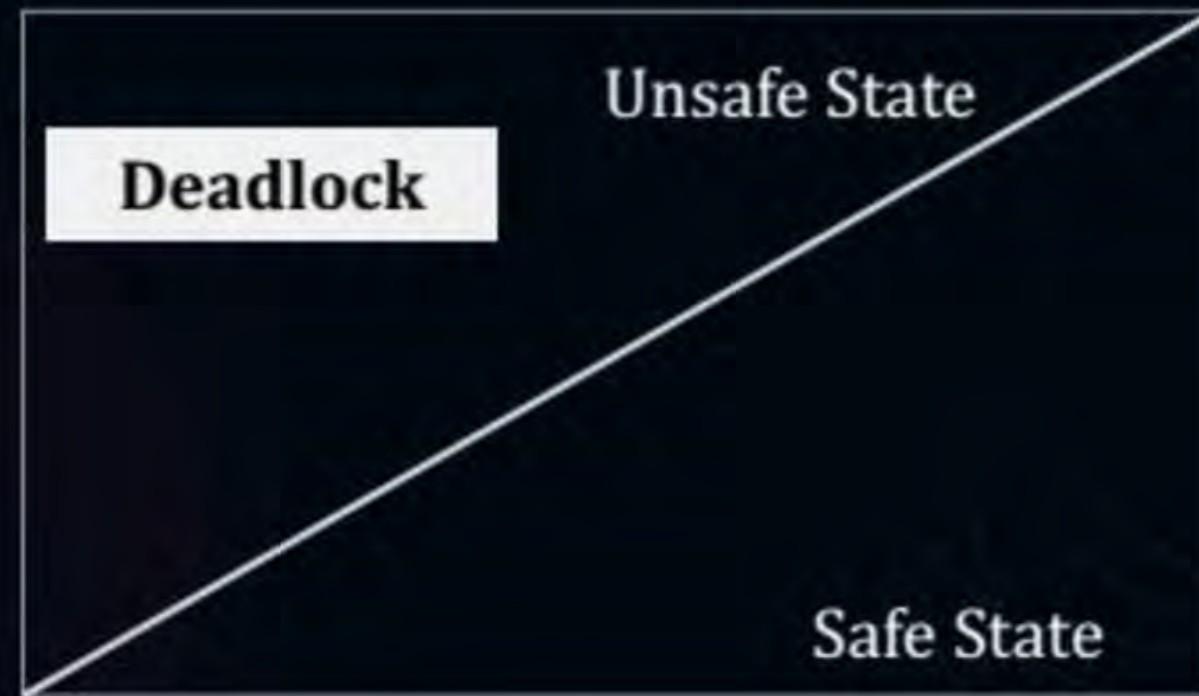
In deadlock avoidance, the OS tries to keep system in safe state

↓
no possibility of deadlock



Topic : Deadlock Avoidance

In deadlock avoidance, the OS tries to keep system in safe state



Process when arrives
announces to os that
↓
what all resources
it will need for execution



Topic : Deadlock Avoidance

In deadlock avoidance, the request for any resource will be granted if the resulting state of the system doesn't cause deadlock in the system.

→ make system in unsafe state



Topic : Banker's Algorithm

The banker's algorithm is a resource allocation and deadlock avoidance algorithm that tests for safety

Banker's algo
→ safety algo
→ Resource request algo



Topic : Banker's Algorithm

Process	Allocation	Max	(free) Available	$need = max - allocation$
P1	1	3	1	2 ✓
P2	5	8	After P3 4	3 ✓
✓ P3	3	4	After P1 5	1 ✓
P4	2	7	After P2 10 After 12	5 ✓

1. find a process P_i for which $need_i \leq Available$
 $available = available + allocation_i$

} all processes executed
↓

safe state

safe sequence $\Rightarrow \langle P3, P1, P2, P4 \rangle$

sequence:- $\langle P_3, P_1, P_4, P_2 \rangle$

$\langle P_3, P_4, P_1, P_2 \rangle$

$\langle P_3, P_2, P_1, P_4 \rangle$

$\langle P_3, P_2, P_4, P_1 \rangle$

safe or not?

safe

unsafe

safe

safe



2 mins Summary

Topic

Deadlock

Topic

Deadlock Prevention

Topic

Deadlock Avoidance





Happy Learning

THANK - YOU

